

DISPENSA DELLA LEZIONE

Form completi e array nello stato

Gestione dell'invio di un form con onSubmit e aggiornamento immutabile
degli array in React.



CAPITOLO 1

onSubmit e gestione del form

NOTA

L'evento `onSubmit` viene eseguito quando un form viene inviato.

In React serve per raccogliere i dati dei campi e gestirli in una funzione, senza lasciare al browser il comportamento predefinito.

Un form non è solo un insieme di input separati. È un blocco unico che ha uno scopo preciso, per esempio registrare un utente, aggiungere un prodotto o inviare un messaggio. Per questo motivo React permette di ascoltare l'evento `onSubmit` direttamente sul tag `form`.

Quando il form viene inviato, il browser prova normalmente a ricaricare la pagina. In un'applicazione React questo comportamento non è utile, perché farebbe perdere lo stato corrente. Per evitarlo si usa il metodo `event.preventDefault()`, che blocca il comportamento automatico del browser e lascia a React il controllo dell'operazione.

In pratica si scrive una funzione, ad esempio `handleSubmit`, e la si collega al form con `onSubmit={handleSubmit}`. Dentro quella funzione si leggono i valori già presenti nello stato, si eseguono eventuali controlli e poi si decide cosa fare, per esempio stampare i dati in console oppure aggiornare un array.

esempio

```
import { useState } from 'react';

function App() {
  const [nome, setNome] = useState('');

  function handleSubmit(event) {
    event.preventDefault();
    console.log('Nome inviato:', nome);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Nome</label>
      <input
        type="text"
        value={nome}
        onChange={(event) => setNome(event.target.value)}
      />
      <button type="submit">Invia</button>
    </form>
  );
}
```

CAPITOLO 2

Gestire più campi nello stesso form

NOTA

Quando un form contiene più input, ogni campo può essere collegato a una parte dello stato.

Al submit, tutti i valori vengono letti insieme e trattati come dati di un unico form.

Un form reale contiene spesso più informazioni, ad esempio nome ed email. In questo caso ogni input continua ad avere il proprio `value` e il proprio `onChange`, ma il submit avviene una sola volta sul form completo. Questo approccio mantiene ordinata la logica: i campi aggiornano lo stato durante la scrittura, mentre `onSubmit` si occupa del momento finale dell'invio.

È importante capire la differenza tra questi due momenti. `onChange` reagisce ogni volta che il valore cambia, quindi serve per tenere aggiornato lo stato. `onSubmit`, invece, interviene quando l'utente conferma il form, quindi serve per usare quei dati insieme.

esempio

```
import { useState } from 'react';

function App() {
  const [nome, setNome] = useState('');
  const [email, setEmail] = useState('');

  function handleSubmit(event) {
    event.preventDefault();
    console.log('Nome:', nome);
    console.log('Email:', email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Nome</label>
      <input
        type="text"
        value={nome}
        onChange={(event) => setNome(event.target.value)}
      />

      <label>Email</label>
      <input
        type="text"
        value={email}
        onChange={(event) => setEmail(event.target.value)}
      />

      <button type="submit">Invia</button>
    </form>
  );
}
```

CAPITOLO 3

Immutabilità dello stato

NOTA

In React lo stato non si modifica direttamente.

Quando un dato deve cambiare, si crea un nuovo valore e lo si passa alla funzione di aggiornamento dello stato.

Immutabilità significa che il valore salvato nello stato non deve essere cambiato direttamente. Se nello stato c'è un array, non si deve aggiungere o togliere elementi modificando quell'array originale. Bisogna invece costruire un nuovo array e usare il setter di `useState` per sostituire il vecchio valore.

Questo principio è importante perché React controlla i cambiamenti di stato per capire quando aggiornare l'interfaccia. Se si modifica direttamente il dato esistente, il cambiamento può diventare poco chiaro e il comportamento dell'applicazione può risultare scorretto o difficile da gestire.

Per questo motivo vanno evitati aggiornamenti diretti come riassegnazioni manuali dell'array originale. L'idea corretta è: partire dal valore attuale, creare una copia aggiornata e passare quella copia a `setState`.

esempio

```
import { useState } from 'react';

function App() {
  const [studenti, setStudenti] = useState(['Anna', 'Luca']);

  function aggiungiStudiante() {
    const nuovoArray = [...studenti, 'Marco'];
    setStudenti(nuovoArray);
  }

  return (
    <div>
      <button onClick={aggiungiStudiante}>Aggiungi</button>
      {studenti.map((studente, index) => (
        <p key={index}>{studente}</p>
      ))}
    </div>
  );
}
```

CAPITOLO 4

Aggiungere elementi a un array

NOTA

Per aggiungere un elemento a un array nello stato si crea un nuovo array che contiene i vecchi elementi più quello nuovo.

Lo strumento più semplice, a questo livello, è lo spread operator dentro un nuovo array.

Quando si vuole salvare un nuovo elemento, per esempio il testo scritto in un input, si prende l'array attuale e lo si ricopia dentro un nuovo array. Alla fine si aggiunge il nuovo valore. In questo modo l'array originale resta intatto e React riceve un nuovo stato completo.

Questo schema è molto utile quando un form serve ad aggiungere elementi a una lista. L'utente compila l'input, preme il pulsante di invio, la funzione `handleSubmit` blocca il refresh della pagina e costruisce un nuovo array con il vecchio contenuto più il nuovo elemento.

esempio

```
import { useState } from 'react';

function App() {
  const [attività, setAttività] = useState(['Studiare']);
  const [nuovaAttività, setNuovaAttività] = useState('');

  function handleSubmit(event) {
    event.preventDefault();
    setAttività([...attività, nuovaAttività]);
    setNuovaAttività('');
  }

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <label>Nuova attività</label>
        <input
          type="text"
          value={nuovaAttività}
          onChange={(event) => setNuovaAttività(event.target.value)}
        />
        <button type="submit">Aggiungi</button>
      </form>

      {attività.map((attivitàSingola, index) => (
        <p key={index}>{attivitàSingola}</p>
      ))}
    </div>
  );
}
```

CAPITOLO 5

Rimuovere elementi da un array

NOTA

Per rimuovere un elemento da un array nello stato non si cancella direttamente il dato esistente.

Si crea invece un nuovo array che contiene solo gli elementi da mantenere.

Per togliere un elemento si usa spesso `filter()`. Questo metodo crea un nuovo array prendendo solo gli elementi che rispettano una condizione. Il vantaggio è che non modifica l'array originale, quindi rispetta l'immutabilità dello stato.

Se, per esempio, si vuole eliminare un elemento in base alla sua posizione, si può confrontare l'indice dell'elemento corrente con l'indice da rimuovere. Tutti gli elementi diversi da quell'indice restano nel nuovo array, mentre quello scelto viene escluso.

esempio

```
import { useState } from 'react';

function App() {
  const [frutti, setFrutti] = useState(['Mela', 'Pera', 'Banana']);

  function rimuoviFrutto(indiceDaRimuovere) {
    const nuovoArray = frutti.filter((frutto, index) => {
      return index !== indiceDaRimuovere;
    });

    setFrutti(nuovoArray);
  }

  return (
    <div>
      {frutti.map((frutto, index) => (
        <div key={index}>
          <p>{frutto}</p>
          <button onClick={() => rimuoviFrutto(index)}>Rimuovi</button>
        </div>
      ))}
    </div>
  );
}
```